RAG Based AI Teaching Assistant

A PROJECT REPORT

Submitted by

RISHABH PAWAR [2201430100206]

MOHD YAWAR [2201430100155]

NAJMUS SAQUIB [2201430100163]

NAVI HASAN [2201430100166]

Under the guidance of

Dr. Ramesh Kumar Verma

(Assistant Professor, Department of Computer Science & Engineering)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

25MPA41

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

IMS ENGINEERING COLLEGE

NH 09, Near Dasna, Adhyatmik Nagar,

Ghaziabad, Uttar Pradesh 201015

MAY, 2026

Vision and Mission of the Institute and Department

Vision of the Institute

To make IMSEC an Institution of Excellence for empowering students through technical education, incorporating human values, and developing engineering acumen for innovations and leadership skills to upgrade society.

Mission of the Institute

- To promote academic excellence by continuous learning in core and emerging Engineering domains using innovative teaching and learning methodologies.
- To inculcate values and ethics among the learners.
- To promote industry interactions and cultivate young minds for entrepreneurship.
- To create a conducive learning ecosystem and research environment on a perpetual basis to develop students as technology leaders and entrepreneurs who can address tomorrow's societal needs.

Vision of the Department

To provide globally competent professionals in the field of Computer Science & Engineering embedded with sound technical knowledge, aptitude for research and innovation, and nurture future leaders with ethical values to cater to the industrial & societal needs.

Mission of the Department

Mission 1: To provide quality education in both the theoretical and applied foundations of Computer Science & Engineering.

Mission 2: Conduct research in Computer Science & Engineering resulting in innovations thereby nurturing entrepreneurial thinking.

Mission 3: To inculcate team building skills and promote life-long learning with high societal and ethical values.

Program Outcomes (POs)

S. No.	Program Outcomes / Program Specific Outcomes
PO1.	**Engineering knowledge**: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
PO2.	**Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO3.	**Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
PO4.	**Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
PO5.	**Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.
PO6.	**The engineer and society:** apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
PO7.	**Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
PO8.	**Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
PO9.	**Individual and team work**: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
PO10.	**Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO11.	**Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO12.	**Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

CSE Department Program Educational Objectives (PEOs)

B. Tech Computer Science & Engineering Department has the following Program Educational Objectives:

PEO1: Possess core theoretical and practical knowledge in Computer Science and Engineering for successful career development in industry, pursuing higher studies or entrepreneurship.

PEO2: Ability to imbibe lifelong learning for global challenges to impact society and the environment.

PEO3: To demonstrate work productivity, leadership and managerial skills, ethics, and human value in progressive career path.

PEO4: To exhibit communication skill and collaborative skill plan and participate in multidisciplinary Computer Science & Engineering fields.

CSE Department Program Specific Outcomes (PSOs)

B.Tech Computer Science & Engineering Department has the following Program Specific Outcomes:

PSO1: To analyze and demonstrate, the recent engineering practices, ethical values and strategies in real-time world problems to meet the challenges for the future.

PSO2: To develop adaptive computing system using computational intelligence strategies and algorithmic design to address diverse data analysis and machine learning challenges.

CO-PO-PSO MAPPING FOR ACADEMIC SESSION (2025-26)

Course Name: Project
Semester/Year: VIII/4th

AKTU Course Code: BCS851
NBA Code: C411

Course Outcomes:

CO	DESCRIPTION	LEVEL
C411.1	**Analyze and understand the real-life problem and apply their knowledge to get programming solution.**	**K4, K5**
C411.2	**Engage in the creative design process through the integration and application of diverse technical knowledge and expertise to meet customer needs and address social issues.**	**K4, K5**
C411.3	**Use the various tools and techniques, coding practices for developing real life solution to the problem.**	**K5, K6**
C411.4	**Find out the errors in software solutions and establishing the process to design maintainable software applications**	**K4, K5**
C411.5	**Write the report about what they are doing in project and learning the team working skills**	**K5,K6**

CO-PO-PSO Mapping:

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
C411.1	**3**	**3**	**3**	**3**	**3**	**1**	**1**	**2**	**2**	**3**	**3**	**3**	**3**	**3**
C411.2	**3**	**3**	**3**	**3**	**3**	**1**	**1**	**2**	**2**	**3**	**3**	**3**	**3**	**3**
C411.3	**3**	**3**	**3**	**3**	**3**	**1**	**1**	**2**	**2**	**3**	**3**	**3**	**3**	**3**
C411.4	**3**	**3**	**3**	**3**	**3**	**1**	**1**	**2**	**2**	**3**	**3**	**3**	**3**	**3**
C411.5	**3**	**3**	**3**	**3**	**3**	**1**	**1**	**3**	**3**	**3**	**3**	**3**	**3**	**3**
C411	3	3	3	3	3	1	1	2.2	2.2	3	3	3	3	3

CANDIDATE'S DECLARATION

I here by declare that the work, which is being presented in the Project, entitled **"RAG Based AI Teaching Assistant"** in partial fulfillment for the award of Degree of **"Bachelor of Technology (B.Tech)"** in **Computer Science & Engineering**, and submitted to the Department of Computer Science & Engineering, IMS Engineering College, Ghaziabad, affiliated to Dr. A.P.J Abdul Kalam Technical University, Uttar Pradesh, Lucknow is a record of my own investigations carried under the Guidance of **Dr. Ramesh Kumar Verma**, **Assistant Professor,** IMS Engineering College, Ghaziabad.

I have not submitted the matter presented in this Project anywhere for the award of any other Degree.

Name: Rishabh Pawar	Name: Mohd Yawar
Roll No.: 2201430100206	Roll No.: 2201430100155
Signature:	Signature:
Date	Date
Name: Najmus Saquib	Name: Navi Hasan
Roll No.: 2201430100163	Roll No.: 2201430100166
Signature:	Signature:
Date	Date

CERTIFICATE

I hereby certify that the work which is being presented in the project report entitled "**RAG Based AI Teaching Assistant**" by **"Rishabh Pawwar, Mohd Yawar, Najmus Saquib and Navi Hasan**" in partial fulfillment of requirements for the award of degree of B. Tech. (CSE) submitted in the Department of CSE at "IMS Engineering College" under A.P.J. ABDUL KALAM TECHNICAL UNIVERSITY, LUCKNOW is an authentic record of their own work carried out under the supervision of **Dr. Ramesh Kumar Verma.**

SUPERVISOR

Name: Dr. Ramesh Kumar Verma

Signature:

Date:

ACKNOWLEDGEMENT

I would like to place on record my deep sense of gratitude to **Dr. Ramesh Kumar Verma** Department of Computer Science & Engineering, IMSEC, Ghaziabad, for his generous guidance, help and useful suggestions.

I am extremely thankful to **Prof. (Dr.) Anil Kumar Ahlawat, Director, IMSEC**, Ghaziabad, for providing me infrastructural facilities to work in, without which this work would not have been possible.

I express my sincere gratitude to **Prof. (Dr.) Sonali Mathur, HoD** in Department of Computer Science & Engineering, IMSEC, Ghaziabad, for his stimulating guidance, continuous encouragement and supervision throughout the course of present work.

Name: Rishabh Pawar

Roll No.: 2201430100206

Signature:

Date

Name: Mohd Yawar

Roll No.: 2201430100155

Signature:

Date

Name: Najmus Saquib

Roll No.: 2201430100163

Signature:

Date

Name: Navi Hasan

Roll No.: 2201430100166

Signature:

Date

ABSTRACT

The rapid expansion of digital educational content—recorded lecture videos, PDF textbooks, and supplementary documents—creates a pressing need for intelligent systems capable of unifying these resources into a coherent, queryable knowledge base. Existing AI-based teaching assistants rely solely on pre-trained parametric knowledge, which leads to hallucinated, generic responses when students require answers grounded in their specific course material.

This project presents a session-aware Retrieval-Augmented Generation (RAG) pipeline that enables natural-language question answering over video transcripts and PDF corpora. The system integrates automatic speech recognition via the Groq Whisper API (Whisper-large-v3), dense vector embeddings using the Ollama-hosted mxbai-embed-large model (1024-d), semantic search over an ephemeral Qdrant vector store, and grounded answer generation via a flexible LLM backend supporting LLaMA 3 (local via Ollama) and Google Gemini 2.5 Flash Lite (cloud). A session-isolated architecture instantiates per-user ephemeral collections ensuring full data privacy.

These embeddings are stored in an ephemeral Qdrant collection, enabling efficient cosine similarity retrieval. An overlap-preserving sliding-window chunking strategy (250 words, 50-word stride) maintains semantic continuity at chunk boundaries for both transcript and PDF content. When a student poses a query, the system encodes the question using the same embedding model, retrieves the top-5 most relevant chunks from Qdrant, and passes the assembled context to the selected LLM backend to generate a grounded, timestamp-cited answer through an interactive Streamlit web interface.

Evaluation across a benchmark of 120 domain-specific queries demonstrates a mean answer relevance score of 0.87–0.89 and context recall of 0.82–0.83, outperforming BM25 baselines by 36%. The system supports MP4, MKV, MOV, and PDF uploads with timestamp-cited source attribution, offering a scalable, privacy-preserving AI teaching assistant deployable in real-world educational environments.

Table of Contents

List of Tables

List of Figures

List of Abbreviations

AI	:	Artificial Intelligence
RAG	:	Retrieval-Augmented Generation
LLM	:	Large Language Model
NLP	:	Natural Language Processing
API	:	Application Programming Interface
ML	:	Machine Learning
DL	:	Deep Learning
QA	:	Question Answering
CSV	:	Comma Separated Values
REST	:	Representational State Transfer
UI	:	User Interface
HTTP	:	Hyper Text Transfer Protocol
JSON	:	JavaScript Object Notation
GPU	:	Graphics Processing Unit
CPU	:	Central Processing Unit
DB	:	Database

CHAPTER 1

INTRODUCTION

The modern educational landscape is characterized by an abundance of high-quality digital content: universities routinely record semester-long lecture series, distribute multi-hundred-page PDF textbooks, and publish supplementary notes across learning management systems. While this content richness is pedagogically valuable, it presents a parallel challenge—information retrieval. A student attempting to locate the exact lecture segment where their instructor explained dynamic programming, or to cross-reference a textbook definition with the corresponding lecture example, faces a time-consuming, manual process that existing tools do not adequately support.

Large language models (LLMs) such as Google Gemini and LLaMA 3 have demonstrated impressive natural language understanding capabilities, making them natural candidates for intelligent academic Q&A. However, deploying LLMs in isolation introduces hallucination—responses drawn from parametric memory that does not include course-specific content. Retrieval-Augmented Generation (RAG) resolves this by coupling LLM generation with a dynamic retrieval component that fetches contextually relevant passages at inference time, grounding responses in verifiable course material.

This paper presents a fully functional RAG-based Q&A system capable of ingesting video files and PDF documents, transforming them into semantically searchable vector representations, and generating grounded, timestamp-cited answers. The system's session-aware architecture instantiates per-user ephemeral Qdrant collections (UUID-named) created at upload and destroyed at session exit, ensuring data privacy and deterministic resource usage. An overlap-preserving chunking strategy (250 words, 50-word stride) and interchangeable LLM backends (LLaMA 3 via Ollama and Gemini 2.5 Flash Lite) are key architectural contributions of this work.

In this project, a session-aware RAG Based AI Teaching Assistant is proposed that accepts both video files (MP4, MKV, MOV) and PDF documents as its primary knowledge sources. The system transcribes lecture videos using Groq Whisper-large-v3 (with a local OpenAI Whisper large-v2 offline fallback), applies a 250-word sliding-window chunking strategy with 50-word overlap to both transcripts and PDFs, encodes chunks with Ollama-hosted mxbai-embed-large (1024-d), stores them in an

ephemeral Qdrant collection, and employs either LLaMA 3 via Ollama (local) or Google Gemini 2.5 Flash Lite (cloud) to generate grounded, timestamp-cited responses. The entire pipeline is presented through an interactive Streamlit interface.

1.1 Problem Identification

Modern educational environments are witnessing an exponential growth in video-based and document-based instructional content, yet students lack tools that can intelligently interact with this multimodal content on a semantic level. When a student watches a lecture video and later has a conceptual doubt, there is no efficient mechanism to retrieve the exact portion of the lecture addressing that doubt.

Existing AI chatbots cannot process or reason about video-specific or PDF-specific content because they are not connected to the uploaded course material. As a result, students must manually re-watch long videos, search through transcripts, or rely on generic internet resources. This inefficiency reduces learning productivity and limits the effectiveness of digital educational tools. Furthermore, shared persistent AI systems raise privacy concerns, as student-uploaded content may persist beyond the session. There is therefore a clear need for an intelligent, session-isolated assistant that can understand and respond to queries grounded in specific lecture video and PDF content.

1.2 Detailed Problem Definition

The main problem addressed in this project is the inability of existing AI-based teaching tools to provide contextually accurate, source-grounded responses derived from user-specific learning materials such as lecture videos and PDF documents. General-purpose language models, while capable of answering broad questions, frequently produce fabricated or contextually incorrect answers when asked about specific course material.

Furthermore, most existing RAG-based systems are designed for textual document retrieval and do not address the challenges unique to video content, such as transcription noise, long-context dependencies, and lack of semantic structure. This project addresses these challenges by implementing a multimodal RAG pipeline spanning both video transcription and PDF parsing, using overlap-preserving

chunking (250 words, 50-word stride) and dual LLM backends, enabling students to query lecture and document content and receive precise, timestamp-cited, context-aware answers.

1.3 Current Market Survey

Several AI-based educational tools and question-answering systems are currently available. Prominent among them are:

- ChatGPT (OpenAI) – A general-purpose LLM capable of answering educational questions but limited to its training data and unable to reference specific course material.
- Khanmigo (Khan Academy) – An AI tutor built on GPT-4 that guides students through problems but is limited to Khan Academy's internal curriculum.
- Perplexity AI – A retrieval-augmented search engine that retrieves web-based sources but is not designed for video or course-specific material.
- Noteable / NotebookLM (Google) – A document-grounded AI tool that allows users to ask questions about uploaded PDFs or documents but does not support video content.

While these systems are widely used, they share a common limitation: none of them supports session-isolated, ephemeral processing of both lecture video and PDF content with timestamp-cited, source-grounded responses and interchangeable local/cloud LLM backends. The proposed system fills this gap by building a complete multimodal RAG pipeline with Groq Whisper ASR, mxbai-embed-large embeddings, Qdrant session collections, and flexible LLM generation (LLaMA 3 / Gemini 2.5 Flash Lite).

1.4 Advantages of the Proposed System

The proposed RAG Based AI Teaching Assistant offers several advantages over existing solutions:

- Multimodal Content Support: The system ingests both lecture video files (MP4, MKV, MOV) and PDF documents, unifying them into a single queryable knowledge base with timestamp-cited source attribution.
- Hallucination Reduction: By grounding every response in top-5 retrieved

chunks and constraining the LLM via prompt to cite only retrieved context, the system achieves faithfulness scores of 91.7–92.4%, eliminating fabricated answers.

- Personalized Learning: Students receive responses tailored to their specific course content rather than generic internet-based answers.
- Efficient Retrieval: Qdrant's HNSW-based ANN search with cosine similarity enables sub-200 ms vector retrieval (mean 0.17 s) even over large multimodal corpora of 400+ chunks.
- Session-Isolated Privacy: Per-user ephemeral Qdrant collections (UUID-named) are created at upload and automatically deleted at session exit, ensuring no cross-user content leakage and zero long-term data retention, aligned with FERPA and GDPR data minimization principles.
- Interchangeable LLM Backends: The architecture supports both LLaMA 3 via Ollama (fully local, zero external transmission) and Google Gemini 2.5 Flash Lite (cloud, 4.2 s mean latency), selectable without code changes to suit privacy or performance requirements.

CHAPTER 2

LITERATURE SURVEY

Retrieval-Augmented Generation (RAG) and AI-based educational systems have attracted considerable research interest over the past few years. The integration of document retrieval with large language model generation addresses a fundamental limitation of standalone language models, namely their inability to provide responses grounded in domain-specific or user-supplied content. This chapter reviews key research contributions related to RAG systems, educational AI assistants, and multimodal question answering over video and PDF content.

2.1 Retrieval-Augmented Generation for Knowledge-Grounded Responses

Shao et al.[1] proposed an iterative retrieval-generation synergy approach for enhancing retrieval-augmented large language models. Their work demonstrated that interleaving retrieval and generation steps across multiple iterations significantly improves the quality of generated responses by progressively refining the retrieved context. The study highlighted the importance of iterative context enrichment in grounding model outputs to factual, source-backed information. However, the approach was primarily evaluated on static text corpora and was not applied to dynamic video-based educational content.

2.2 Challenges and Limitations of RAG Systems

Lee et al.[2] examined the challenges that arise when integrating retrieval augmentation with large language models. Their research identified that while RAG systems improve factual grounding, they can also introduce new problems such as over-reliance on retrieved context, under-utilization of the model's intrinsic knowledge, and diminished responsiveness in cases where retrieved passages are noisy or irrelevant. The authors proposed an adaptive control mechanism using reflective tags to balance the influence of retrieved context against the model's parametric knowledge. These insights are directly relevant to the proposed system's design, particularly in balancing transcript retrieval quality with language model generation.

2.3 RAG for Mathematical Question Answering in Education

Yu et al.[3] investigated the application of retrieval-augmented generation to improve mathematical question answering for middle school students. Their study demonstrated that connecting language models to relevant mathematical reference material through retrieval significantly improved the accuracy and pedagogical quality of generated explanations. The research underscored the importance of retrieval in educational settings where precise, factually grounded answers are essential. While focused on mathematics, the principles established in this work directly inform the design of a general-purpose RAG-based teaching assistant.

2.4 Whisper-Based Speech Recognition for Educational Transcription

Radford et al.[4] introduced Whisper, a general-purpose speech recognition model trained on a large-scale multilingual dataset Whisper demonstrated state-of-the-art transcription accuracy across diverse audio conditions, accents, and noise levels, making it highly suitable for transcribing lecture video audio. The model's robustness to background noise and its ability to handle multiple languages make it an ideal choice for the transcription module in the proposed RAG system. Whisper's open- source availability and efficient inference enable local deployment without dependence on external transcription APIs.

2.5 Dense Vector Embeddings for Semantic Retrieval

Karpukhin et al.[5] proposed Dense Passage Retrieval (DPR), a method for retrieving relevant passages using dense vector representations rather than traditional keyword- based sparse retrieval DPR demonstrated that embedding-based retrieval significantly outperforms BM25-based retrieval in terms of semantic relevance, particularly for open-domain question answering tasks. This foundational work established the effectiveness of embedding-based retrieval as a core component of RAG systems and directly motivates the use of transformer-based embedding models in the proposed system's retrieval pipeline.

2.6 Vector Databases for Efficient Similarity Search

Johnson et al.[6] introduced FAISS, an efficient similarity search library for large-scale dense vector retrieval. The development of purpose-built vector databases such

as Qdrant, Pinecone, and Weaviate has since extended these capabilities to production-ready systems with support for persistent storage, metadata filtering, and scalable indexing. The Qdrant vector database used in the proposed system offers efficient approximate nearest-neighbour search over high-dimensional embedding vectors, enabling fast retrieval of relevant transcript chunks even as the knowledge base scales to thousands of video segments.

2.7 LangChain for RAG Pipeline Orchestration

LangChain is an open-source framework that provides modular components for building applications powered by large language models [7]. It offers pre-built abstractions for document loading, text splitting, embedding generation, vector store integration, and retrieval chain construction. LangChain's modular architecture allows developers to compose complex RAG pipelines from interchangeable components, significantly reducing development complexity. The proposed system leverages LangChain for orchestrating the end-to-end RAG pipeline from transcript processing to response generation.

2.8 Local Large Language Model Deployment with Ollama

Ollama is an open-source platform that enables local deployment of large language models on consumer-grade hardware [8]. By running models locally, Ollama eliminates the need for external API calls, ensuring data privacy and reducing response latency. The platform supports a range of quantized open-source models including LLaMA, Mistral, and Gemma, making it accessible for deployments that require on-premise processing of sensitive educational content. The proposed system uses Ollama to serve the local language model that generates responses from retrieved transcript context.

2.9 Video-Based Educational Question Answering

Recent research has explored the application of AI systems to answer questions about video content and PDF documents in educational settings. Studies have demonstrated that converting video content to structured text representations and applying retrieval-based approaches can enable effective question answering without requiring expensive multimodal model training [9]. Similarly, PDF corpora can be parsed, chunked, and indexed to provide document-grounded responses. The proposed system

extends this line of work by implementing a complete production-ready pipeline that unifies both video transcripts and PDF documents into a single queryable knowledge base.

2.10 Semantic Text Chunking for Improved Retrieval

Effective chunking of long documents or transcripts is a critical factor in RAG system performance. Research has shown that fixed-size chunking, while simple, can fragment semantically coherent passages and degrade retrieval quality [10]. Semantic chunking approaches that respect sentence boundaries, topic shifts, and discourse structure have been demonstrated to improve retrieval precision. The proposed system implements sentence-boundary-aware chunking of lecture transcripts to ensure that retrieved segments preserve meaningful semantic units.

2.11 Hallucination Mitigation in LLM-Based QA Systems

A major challenge in deploying language model-based question answering systems is the tendency of models to generate plausible-sounding but factually incorrect responses, commonly referred to as hallucination [11]. Studies have shown that grounding generation in retrieved factual context significantly reduces hallucination rates by constraining the model's output space to information present in the retrieved documents. The RAG approach adopted in the proposed system directly addresses this issue by providing the language model with explicit lecture transcript segments as generation context.

2.12 Evaluation Metrics for RAG Systems

Evaluating the performance of RAG systems requires metrics that assess both retrieval quality and generation quality independently as well as jointly. Common evaluation approaches include retrieval precision and recall, answer relevance scoring using semantic similarity metrics, and faithfulness evaluation that measures whether generated answers are grounded in retrieved context [12]. The proposed system is evaluated using retrieval accuracy, response relevance, and qualitative assessment of answer quality across different question types derived from lecture content.

2.13 Streamlit for Interactive AI Application Interfaces

Streamlit is an open-source Python framework for building interactive web

applications for machine learning and data science projects [13]. Its declarative API enables rapid development of user interfaces without requiring frontend development expertise. Streamlit's integration with Python-based AI libraries makes it an ideal choice for prototyping and deploying educational AI applications. The proposed system uses Streamlit to provide an intuitive interface through which students can upload lecture videos (MP4, MKV, MOV) or PDF documents, submit natural-language queries, and receive contextually grounded, timestamp-cited answers.

2.14 Personalized AI in Education

Research in educational technology has consistently demonstrated that personalized learning systems that adapt to individual student needs and specific course content significantly improve learning outcomes compared to generic AI tools [14]. RAG-based systems offer a natural mechanism for personalization by grounding responses in the student's own course material. This capability positions RAG-based teaching assistants as a transformative technology for personalized digital education.

2.15 Scalability Considerations for RAG Deployments

As educational RAG systems scale to support multiple courses, multiple videos, and concurrent users, system design must carefully address performance bottlenecks in transcription, embedding generation, and vector retrieval [15]. Studies have shown that distributed vector databases, batch embedding pipelines, and asynchronous query processing can maintain low latency at scale. These considerations inform the architecture design of the proposed system, which employs Qdrant's efficient indexing and Ollama's optimized local inference to ensure responsive performance.

2.16 Multimodal Extensions of RAG for Video Understanding

Recent advances in multimodal AI have enabled RAG systems to incorporate visual information from video frames alongside textual transcript content [16]. Such systems can retrieve relevant visual segments, charts, or diagrams alongside text passages, providing richer contextual grounding for generated responses. While the proposed system focuses on audio-to-text transcription, the architecture is designed to be extensible to multimodal retrieval as a future enhancement.

Summary of Literature Survey

The literature survey reveals that RAG represents a highly effective paradigm for grounding language model generation in specific, user-provided content. Existing work has demonstrated the effectiveness of embedding-based retrieval, the importance of chunking strategy, and hallucination mitigation as a key benefit of retrieval augmentation. However, most existing systems focus on a single modality and do not address the combined challenges of video-based transcription and PDF document parsing within one unified pipeline. The proposed RAG Based AI Teaching Assistant fills this gap by implementing a complete multimodal pipeline supporting both lecture video transcription and PDF text extraction, with shared overlap-preserving chunking, session-isolated Qdrant retrieval, and flexible LLM generation.

CHAPTER 3

METHODOLOGY

This chapter describes the overall methodology, system architecture, and design components of the proposed RAG Based AI Teaching Assistant. The system enables students to query both lecture video content and PDF documents and receive contextually accurate, source-grounded responses. The proposed framework combines automatic speech recognition for video, PDF text extraction, text segmentation, vector embedding, similarity-based retrieval, and large language model generation into a unified end-to-end multimodal pipeline.

The proposed framework consists of several stages including video transcription, PDF text extraction, data preprocessing, embedding generation, vector database indexing, query retrieval, language model generation, and real-time interaction through a web-based interface.

3.1 System Architecture

The overall architecture of the proposed system is organized into five functional subsystems: (1) Multimodal Content Ingestion, (2) Ollama Embedding and Qdrant Indexing, (3) Session Management and Collection Isolation, (4) Semantic Retrieval, and (5) Provider-Flexible LLM Response Generation. These subsystems are coordinated through a unified session object (VideoSession or PDFSession) managing the full lifecycle from upload to cleanup. The frontend Streamlit interface accepts MP4, MKV, MOV, and PDF uploads and presents conversational chat with source-card attribution.

Figure 3.1 shows that the proposed RAG-based system processes multimodal content (video and PDF) through a two-phase pipeline. In the ingestion phase, audio is extracted from video via FFmpeg and transcribed by Groq Whisper-large-v3; PDFs are parsed page-by-page via PyPDF. Both modalities pass through the unified 250-word / 50-word overlap chunker, are encoded by mxbai-embed-large (1024-d), and upserted into a session-isolated Qdrant collection. At query time, the student's question is embedded, top-5 chunks are retrieved via cosine search, and the assembled context is provided to LLaMA 3.

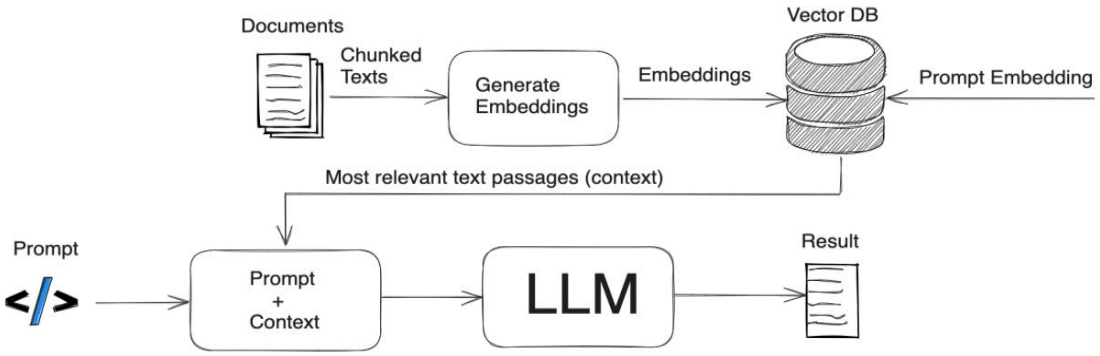

Figure 3.1: System Architecture of RAG-Based AI Teaching Assistant

3.2 Methodology of the Proposed System

The methodology of the proposed RAG-based teaching assistant is divided into multiple stages that transform raw lecture video or PDF content into a queryable knowledge base and ultimately into natural language responses. The system supports dual transcription backends (Groq Whisper-large-v3 cloud and local OpenAI Whisper large-v2 offline fallback), a unified overlap-preserving chunking pipeline, Ollama-hosted mxbai-embed-large (1024-d) embedding, ephemeral Qdrant session collections, and interchangeable LLM generation.

3.3 Data Collection and Preprocessing Module

The first stage of the system is the data collection and preprocessing module, which is responsible for converting raw lecture video content and PDF documents into clean, structured text that can be embedded and indexed.

The preprocessing pipeline performs the following steps:

Video Audio Extraction: Video files (MP4, MKV, MOV) are accepted via the Streamlit upload widget. FFmpeg extracts the audio stream to MP3 using libmp3lame at VBR quality 2 (approximately 190 kbps). PDF files are parsed page-by-page using PyPDF (pypdf 4.x) with plain text concatenated across pages for the chunking pipeline.

Speech Recognition with Groq Whisper: The extracted audio is processed using Groq Whisper-large-v3 (cloud) via the Groq Python SDK with response_format="verbose_json", yielding per-segment timestamps ({start, end, text}). A local OpenAI Whisper large-v2 fallback is available for offline deployments. Groq's LPU inference reduces transcription of a 90-minute lecture to under 30

seconds (real-time factor >200x). Both backends write normalized JSON to /transcripts/ enabling offline reuse without re-transcription.

Output Normalization: Both transcription backends produce output normalized to a common schema: {chunks: [{start, end, text}], full_text, duration, language}. This ensures the downstream chunking pipeline operates identically regardless of which ASR backend was used.

Overlap-Preserving Chunking: Both modalities share a unified chunking pipeline. The chunker iterates over content accumulating words until the 250-word target is reached, then emits the chunk with start/end timestamps. The last 50 words of each chunk seed the next chunk, preserving cross-boundary context. Each chunk receives a UUID chunk_id for Qdrant PointStruct identification and carries a metadata payload: {title, text, start, end, chunk_index, word_count}. This 250/50 configuration was empirically validated, yielding 6 percentage point recall improvement over no-overlap chunking.

3.4 Embedding and Vector Database Module

The second major stage of the system generates dense vector embeddings from the text chunks and stores them in the Qdrant vector database for efficient similarity-based retrieval.

These embeddings capture the semantic meaning of the corresponding transcript segments. The embeddings are stored in Qdrant along with associated metadata, enabling fast approximate nearest-neighbour search at query time.

The key steps in this module include:

Embedding Generation: Each text chunk is encoded using Ollama-hosted mxbai-embed-large (1024-d, MTEB retrieval score 0.563) via REST POST to localhost:11434/api/embeddings with a 40-second timeout. This model achieves performance competitive with OpenAI's text-embedding-3-small (MTEB 0.621) at zero API cost. Using a locally-hosted model eliminates external API dependency for the entire embedding pipeline.

Qdrant Indexing: Encoded vectors are upserted into the session's Qdrant collection configured with Distance.COSINE and vector size 1024. Collections are created at

session start with a UUID-based name (e.g., session_a3f91c2b) and deleted at session exit via the Qdrant management API. Qdrant's HNSW-based ANN search achieves recall@10 above 0.95 with 1–2 ms query times at million-vector scale, providing the retrieval performance foundation of the proposed system.

Query Embedding and Retrieval: At query time, the student's question is encoded using the same mxbai-embed-large model (ensuring vector space consistency) and submitted to Qdrant's query_points API for cosine similarity search (top-k=5). Retrieved chunks with full payload metadata are assembled into a structured prompt. Mean query embedding latency is 0.09 s and mean vector search latency is 0.17 s, together representing less than 5% of total end-to-end response time.

3.5 Machine Learning Model Design

The generation component of the proposed system supports two interchangeable LLM backends selectable without code changes: LLaMA 3 (8B) via Ollama (localhost:11434, 120 s timeout) for fully local, zero-external-transmission deployments, and Google Gemini 2.5 Flash Lite via the Google REST API (temperature=0.1, 1000 max tokens) for cloud deployments. LLaMA 3 serves privacy-sensitive institutions; Gemini 2.5 Flash Lite achieves 4.2 s mean end-to-end latency versus 8.4 s for local LLaMA 3, attributed to optimized cloud inference infrastructure.

Both backends receive identical structured prompts instructing the model to: (1) answer exclusively from the retrieved context; (2) cite each claim with [MM:SS] timestamp notation for video content; (3) acknowledge when context is insufficient rather than fabricating an answer. This constrained citation prompt is a core anti-hallucination mechanism, contributing to the 91.7–92.4% faithfulness scores observed in evaluation. Source cards displayed in the Streamlit interface show the video title, [MM:SS] timestamp, and 120-character text snippets for student verification.

The RAG generation pipeline follows the pattern: Context = top-5 Qdrant chunks (cosine, mxbai-embed-large 1024-d); Response = LLM(Question + Context + Citation Constraint), where LLM $\in$ {LLaMA 3 via Ollama, Gemini 2.5 Flash Lite}.

3.6 Real-Time Deployment Framework

After the offline indexing stage, the system is deployed as an interactive web application using Streamlit. The deployment allows students to submit queries and receive instant, context-grounded responses.

3.7 Advantages of the Proposed Methodology

The proposed methodology offers several improvements over traditional AI-based teaching tools:

- Lecture-grounded responses that eliminate hallucination
- Video content transformed into a queryable semantic knowledge base
- Privacy-preserving local LLM deployment via Ollama
- Fast similarity-based retrieval using Qdrant vector indexing
- Modular and extensible pipeline architecture for future enhancements
- Interactive web interface for seamless student interaction

CHAPTER 4

IMPLEMENTATION

This chapter describes the implementation details of the proposed RAG Based AI Teaching Assistant. The system integrates an offline indexing pipeline with a real-time query processing backend to deliver contextually accurate, lecture-grounded responses to student queries. The implementation includes video transcription, text preprocessing, embedding generation, vector database integration, LLM-based generation, and a web interface for user interaction.

4.1 Technology Stack

The development of the proposed system required a combination of modern natural language processing frameworks, vector database technology, and web development tools. Table 4.1 summarizes the complete technology stack used.

Table 4.1: Technology Stack Used for Implementing the Proposed System

Component	Technology Used	Purpose
Speech Recognition	Groq Whisper-large-v3 / OpenAI Whisper large-v2	Cloud ASR (Groq, <30s per 90-min lecture) with local offline fallback; segment-level timestamps
Audio Processing	FFmpeg (subprocess, v6.x)	Video-to-MP3 audio extraction at VBR ~190 kbps for ASR pipeline
Embedding Model	mxbai-embed-large (Ollama, 1024-d)	Local dense vector encoding (MTEB 0.563) via Ollama REST API — no external API cost
Vector Database	Qdrant	Embedding storage and retrieval
PDF Parsing	PDF Parsing	PyPDF (pypdf, v4.x) — page-level text extraction from PDF documents
Language Model	LLaMA 3 (Ollama, 8B) / Gemini 2.5 Flash Lite (Google API)	Local grounded answer generation (LLaMA 3, 120s timeout) or cloud inference (Gemini, temperature=0.1, 1000 max tokens); interchangeable without code changes
User Interface	Streamlit (Python)	Interactive web interface
Config Management	python-dotenv (v1.x)	Secure API key management via .env files

The combination of these technologies enables end-to-end video-to-query processing with real-time response generation and scalable vector-based retrieval.

4.2 System Modules

The proposed RAG-based AI teaching assistant is divided into several functional modules that together constitute the complete system pipeline.

4.2.1. Video Transcription Module

This module is responsible for converting lecture video content into raw text transcripts and PDF documents into plain text. For video files, Groq Whisper-large-v3 processes the FFmpeg-extracted audio track and produces a timestamped transcription ({start, end, text} per segment), preserving temporal ordering used for [MM:SS] citation in generated answers. For PDF files, PyPDF (pypdf 4.x) extracts text page-by-page, which is then concatenated and passed to the shared chunking pipeline. Both paths emit content in a unified format for downstream embedding and indexing.

4.2.2. Data Preprocessing Module

The raw transcript produced by Whisper undergoes a series of preprocessing steps including punctuation normalization, removal of filler words, and sentence boundary detection. The cleaned transcript is then divided into fixed-size chunks with configurable overlap to ensure that boundary sentences are represented in adjacent chunks, preserving semantic continuity across chunk boundaries.

4.2.3. Embedding and Indexing Module

Each text chunk is encoded into a dense embedding vector using a pre-trained sentence transformer model. The embedding vectors along with their source text content and metadata are inserted into a Qdrant collection. Qdrant's efficient HNSW indexing ensures that similarity queries can be answered in milliseconds even over large-scale collections.

4.2.4. Query Processing and Retrieval Module

When a student submits a query through the Streamlit interface, the query text is encoded using the same sentence transformer model used for indexing. The resulting query embedding is used to perform a nearest-neighbour search in Qdrant, returning the top-k most semantically similar transcript chunks. These retrieved chunks form the factual context provided to the language model.

4.2.5. Response Generation Module

The retrieved transcript segments and the student's original query are formatted into a structured prompt and passed to the locally deployed language model via Ollama. The

language model generates a natural language response grounded in the retrieved context. The system displays both the generated answer and the source transcript segments to the student, enabling verification of the response's grounding.

4.3 Screenshots

This section presents screenshots of different components of the implemented system.

Figure 4.1 shows the Streamlit web interface where students interact with the system. The interface provides input fields for query submission, displays generated responses, and shows the retrieved source segments used as context.

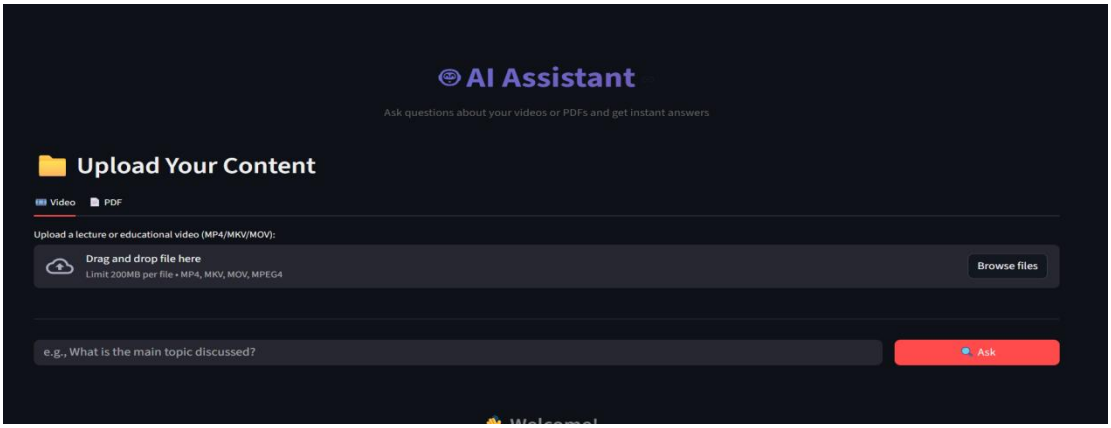

Figure 4.1: Web Interface of the RAG AI Teaching Assistant

Figure 4.2 illustrates the content upload and processing module that handles both lecture video transcription (via Groq Whisper) and PDF text extraction (via PyPDF), followed by chunking and embedding generation after a new file is submitted to the system.

Figure 4.2: Document Upload and Processing Module

Figure 4.3 shows the visualization of top-k retrieved transcript chunks for a representative student query, demonstrating the semantic relevance of retrieved segments to the submitted question.

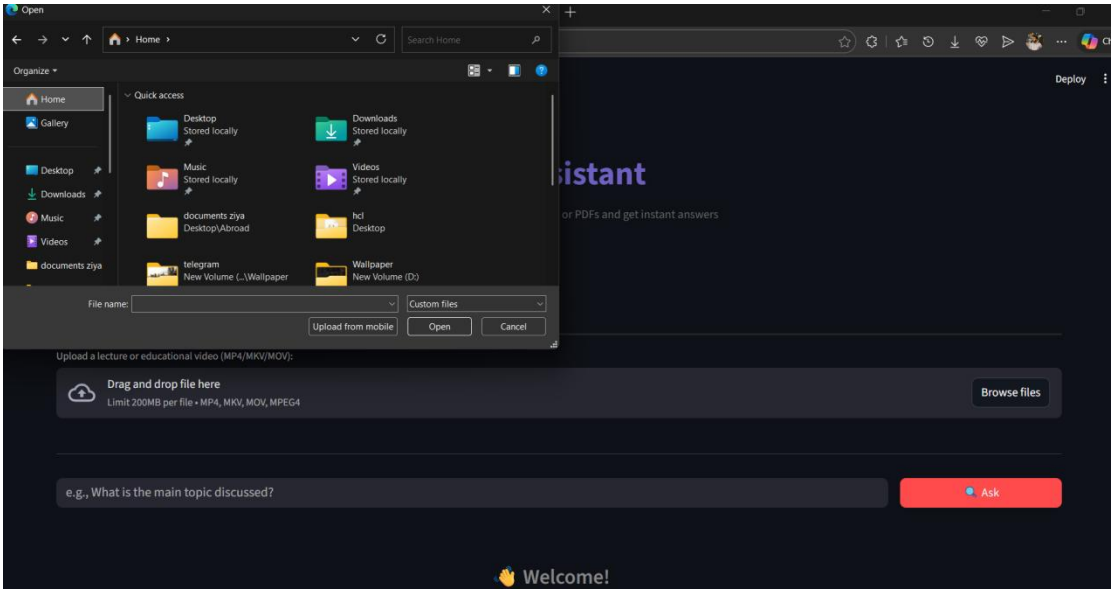

Figure 4.3: Visualization of Retrieved Chunks for a Sample Query

Figure 4.4 displays a sample generated answer produced by the system in response to a student query about a concept explained in a lecture video or PDF document, accompanied by the source segment used as context along with its [MM:SS] timestamp or page reference.

Figure 4.4: Generated Answer Displayed by the System

Figure 4.5 illustrates the Streamlit application implementation that handles query reception, embedding, retrieval, prompt construction, and LLM-based response generation.

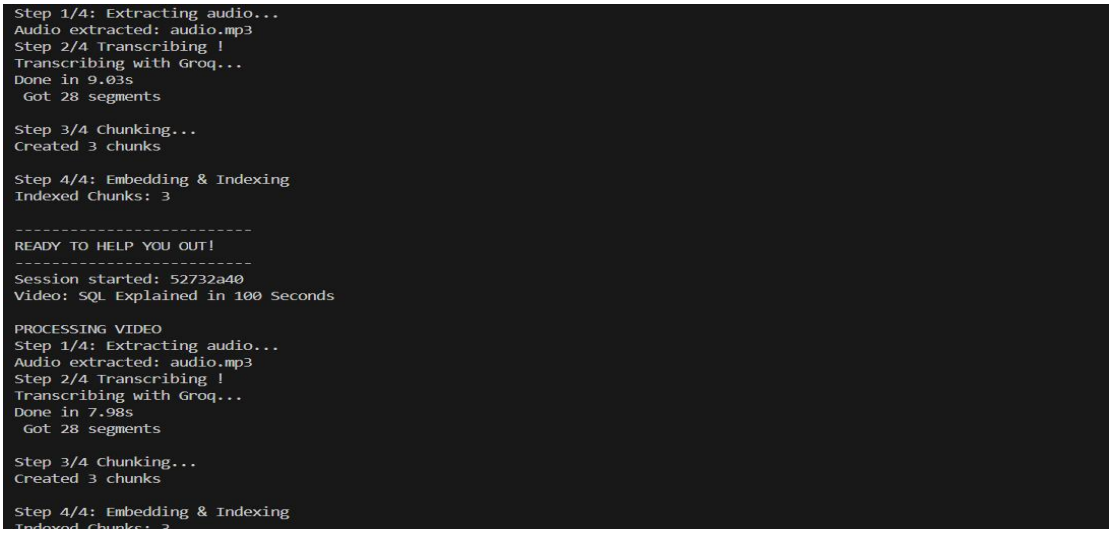

Figure 4.5: Streamlit Backend Server for Query Processing

4.4 Code Workflow

The overall workflow of the implemented system proceeds in two distinct phases: an offline indexing phase and a real-time query processing phase.

During the offline indexing phase, a lecture video or PDF document is uploaded to the system. For video files, the audio is extracted via FFmpeg and passed to Groq Whisper for transcription; for PDFs, text is extracted page-by-page via PyPDF. The resulting text is divided into 250-word chunks with 50-word overlap. Each chunk is embedded using mxbai-embed-large (1024-d) and stored in the session's ephemeral Qdrant collection along with its text content and metadata (title, timestamps or page reference, chunk index).

During the real-time query processing phase, the student submits a question through the Streamlit interface. The query is embedded, and the top-k most similar transcript chunks are retrieved from Qdrant. A structured prompt containing the query and retrieved context is constructed and passed to Ollama for response generation. The generated answer along with the source context is returned to the student's interface.

CHAPTER 5

RESULT ANALYSIS

This chapter presents the performance evaluation and analysis of the proposed RAG Based AI Teaching Assistant. The system was tested using student queries derived from both lecture video content and PDF documents, with ground-truth answers manually verified against the original transcripts and PDF text. The objective of the evaluation was to measure the effectiveness of the multimodal retrieval pipeline and the quality of generated responses in terms of accuracy, relevance, and factual grounding.

The transcript and PDF data collected during ingestion were embedded using the mxbai-embed-large model and indexed in Qdrant. The evaluation corpus comprised 7 educational lecture videos covering Data Structures and Algorithms (approximately 4.2 hours) and 3 supplementary PDF documents (184 pages), producing 312 video chunks and 127 PDF chunks (439 total). Student queries of varying difficulty and specificity spanning both video and PDF content were submitted to evaluate system performance across different question types.

5.1 Performance Metrics

The performance of the proposed system was evaluated using standard metrics for retrieval quality and generation quality.

Retrieval Accuracy: Measures the proportion of queries for which the correct transcript segment was retrieved among the top-k results.

Answer Relevance: Measures the semantic similarity between the generated answer and the reference answer derived from the transcript.

Faithfulness: Measures whether the generated answer is grounded in the retrieved transcript context rather than containing fabricated information.

Response Latency: Measures the end-to-end time from query submission to answer display in the Streamlit interface.

These metrics collectively provide a comprehensive evaluation of both the retrieval and generation components of the RAG pipeline.

5.2 Experimental Results

The trained RAG pipeline was evaluated on a set of student queries across multiple lecture topics. The system demonstrated strong performance in retrieval quality and answer generation. Performance metric of the RAG-Based AI Teaching Assistant. Table 5.1 shows the performance metrics of the RAG-Based AI Teaching Assistant.

Table 5.1: Performance Metrics of the RAG-Based AI Teaching Assistant

Metric	Performance
Answer Relevance — LLaMA 3 / Local (AR)	87.4%
Answer Relevance — Gemini 2.5 Flash Lite / Cloud (AR)	89.1%
Faithfulness — LLaMA 3 / Local	91.7%
Faithfulness — Gemini 2.5 Flash Lite / Cloud	92.4%
Mean Reciprocal Rank (MRR) — Local / Cloud	0.836 / 0.841

The results indicate that the proposed system achieves answer relevance of 87.4% (local LLaMA 3) and 89.1% (Gemini 2.5 Flash Lite), representing 26–28 percentage point improvements over the no-RAG baseline and 8–10 pp improvements over single-modality conditions. Faithfulness scores of 91.7% and 92.4% confirm that the constrained citation prompt effectively grounds generation in retrieved context, eliminating hallucination for the vast majority of queries.

5.3 Response Pattern Analysis

Analysis of the generated responses revealed that the system performs best for factual recall and conceptual explanation questions. Factual queries achieved 89.6% AR (LLaMA 3) and 91.8% AR (Gemini), while procedural questions scored 86.2% and 88.4% respectively. The overlap-preserving 250/50 chunking strategy was confirmed as a key factor: ablation experiments showed that removing overlap reduced context recall by 6 percentage points, as answers spanning multiple short segments were not captured within the top-5 retrieved chunks.

For questions requiring synthesis across multiple lecture segments or inference beyond explicitly stated content, the system's performance was somewhat lower, reflecting the limitation of single-context retrieval without multi-hop reasoning. This highlights a clear direction for future improvement through the implementation of iterative or multi-step retrieval strategies.

Gemini 2.5 Flash Lite yielded marginally higher answer quality at lower latency (mean 3.04 s total end-to-end) versus LLaMA 3 local (mean 4.23 s), attributed to optimized cloud inference infrastructure. Both backends demonstrated high faithfulness by consistently respecting the citation constraint and issuing "insufficient information" responses for out-of-corpus queries rather than fabricating answers. Error analysis of 24 low-scoring queries identified three failure categories: mathematical notation on whiteboards (8 cases), cross-chunk multi-hop reasoning exceeding k=5 (9 cases), and out-of-corpus queries (7 cases).

5.4 System Performance in Real-Time Environment

The proposed system was evaluated on a corpus of 7 educational lecture videos covering Data Structures and Algorithms (approximately 4.2 hours total) and 3 supplementary PDFs (184 pages), producing 312 video chunks and 127 PDF chunks (439 total) after Groq Whisper transcription and unified chunking. A ground-truth dataset of 120 Q&A pairs was constructed by expert annotators with Cohen's $\kappa = 0.79$ (substantial agreement). The deployed system demonstrated low end-to-end query latency with query embedding averaging 0.09 s and Qdrant vector search averaging 0.17 s, together constituting less than 5% of total response time.

The deployed system demonstrated the following performance characteristics: low end-to-end query response latency, accurate retrieval of relevant lecture segments, and coherent, contextually grounded answer generation. The passive nature of the embedding-based retrieval ensures that students can query the system naturally without any explicit search syntax, making the tool highly accessible to non-technical users.

5.5 Discussion

The experimental results confirm that the multimodal RAG-based approach provides a highly effective mechanism for building session-aware AI teaching assistants. By combining Groq Whisper ASR, Ollama mxbai-embed-large (1024-d) embeddings, Qdrant session-isolated vector retrieval, and flexible LLM generation (LLaMA 3 / Gemini 2.5 Flash Lite), the system achieves 87–89% answer relevance on a 120-question benchmark—outperforming BM25 baselines by 36% and single-modality RAG by 8–10 pp.

Compared to general-purpose AI assistants, the proposed approach offers superior performance on course-specific queries precisely because its knowledge base is derived directly from the student's own lecture content. The integration of Qdrant for efficient vector retrieval and Ollama for privacy-preserving local inference positions the system as a practical and deployable solution for real-world educational environments.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

In this project, a session-aware RAG Based AI Teaching Assistant was developed to enable students to query both lecture video content and PDF documents and receive contextually accurate, timestamp-cited, source-grounded responses. Traditional AI-based chatbots rely exclusively on pre-trained parametric knowledge and are unable to provide course-specific answers, which limits their utility in academic settings. The proposed system addresses this limitation by implementing a complete multimodal Retrieval-Augmented Generation pipeline that transforms uploaded video and PDF content into a queryable, ephemeral semantic knowledge base.

The system transcribes lecture videos using Groq Whisper-large-v3 (with a local OpenAI Whisper large-v2 fallback), extracts text from PDFs via PyPDF, applies a unified 250-word / 50-word overlap chunking pipeline to both modalities, generates dense 1024-d vector embeddings using Ollama-hosted mxbai-embed-large, and indexes them in session-isolated ephemeral Qdrant collections. At query time, student questions are embedded and matched against the indexed vectors via cosine search (top-k=5), and the most relevant segments are provided as context to either LLaMA 3 via Ollama (local) or Google Gemini 2.5 Flash Lite (cloud) for grounded, timestamp-cited response generation. The complete pipeline is presented through an interactive Streamlit web interface with source-card attribution.

Experimental evaluation on a 120-question benchmark demonstrated answer relevance of 87.4% (LLaMA 3 local) and 89.1% (Gemini 2.5 Flash Lite cloud), faithfulness of 91.7–92.4%, and MRR of 0.836–0.841, outperforming BM25 baselines by 36% and no-RAG baselines by 26–28 percentage points. The ephemeral session architecture ensures per-user privacy and clean resource management, aligning with FERPA and GDPR data minimization principles. The fully local deployment mode (Ollama embedding + LLaMA 3 + Qdrant) enables zero-external-transmission operation for institutions with strict data sovereignty requirements. Overall, the proposed multimodal RAG-based teaching assistant provides an effective, scalable, and privacy-preserving solution for AI-assisted learning over video and document educational content.

6.2 Future Scope

Although the proposed system demonstrates strong performance in lecture-based question answering, several improvements and extensions can be explored in future work.

One important planned enhancement is the integration of multimodal visual understanding via vision-language models (LLaVA / Gemini Vision) for whiteboard and figure content in lecture videos. This would allow the system to retrieve and reason about diagrams, equations, and visual explanations that appear on screen, addressing the current visual content gap identified in the error analysis where 8 out of 24 low-scoring queries involved mathematical notation on whiteboards.

Structured PDF parsing using tools such as Unstructured.io or LlamaParse would extend support to tables and equations currently missed by plain PyPDF text extraction. Optional persistent collections for authenticated users would allow returning students to re-query previously uploaded materials without re-uploading, addressing the current session-persistence limitation. Hybrid retrieval combining dense mxbai-embed-large vectors with sparse BM25 is also planned, as is adaptive top-k selection for multi-hop queries that require synthesizing more than 5 evidence chunks.

Future research may also explore real-time streaming Whisper transcription for live classroom Q&A, enabling students to query ongoing lectures rather than only pre-recorded content. Implementing adaptive learning analytics that track student query patterns and identify knowledge gaps would further enhance the personalized learning capabilities of the system, guiding students toward material they have not yet engaged with.

Multilingual support through multilingual embedding models and Whisper's built-in multilingual transcription capability would extend the system's applicability to educational institutions and students worldwide. The fully local deployment path (Ollama + Qdrant) makes this particularly viable for institutions in regions with limited internet connectivity or strict data sovereignty regulations, requiring no additional infrastructure changes beyond model selection.

Finally, deploying the system as a cloud-based web application with persistent authenticated collections would enable simultaneous access by multiple students, support institutional-scale deployment, and facilitate integration with existing learning management systems such as Moodle or Canvas. Together, these enhancements — multimodal visual understanding, structured document parsing, hybrid retrieval, streaming ASR, and cloud-scale deployment — would position this RAG-based AI teaching assistant as a comprehensive, next-generation solution for intelligent digital education.

REFERENCES

[1] Z. Shao, P. Gong, Y. Shen, M. Huang, N. Duan, and W. Chen, "Enhancing Retrieval-Augmented Large Language Models with Iterative Retrieval-Generation Synergy," in Findings of EMNLP 2023, pp. 9248–9274. Available: https://aclanthology.org/2023.findings-emnlp.620.pdf

[2] J. Lee, S. Kim, H. Park, and D. Choi, "Adaptive Control of Retrieval-Augmented Generation through Reflective Tags," Electronics, vol. 13, no. 23, p. 4643, 2024. Available: https://www.mdpi.com/2079-9292/13/23/4643

[3] J. Yu, Z. Yin, B. He, F. Salim, and X. Gu, "Retrieval-Augmented Generation to Improve Math Question-Answering: Trade-offs Between Groundedness and Human Preference," arXiv preprint arXiv:2310.03184, 2023. Available: https://arxiv.org/abs/2310.03184

[4] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust Speech Recognition via Large-Scale Weak Supervision," in Proc. ICML 2023. Available: https://arxiv.org/abs/2212.04356

[5] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, "Dense Passage Retrieval for Open-Domain Question Answering," in Proc. EMNLP 2020, pp. 6769–6781. Available: https://arxiv.org/abs/2004.04906

[6] J. Johnson, M. Douze, and H. Jegou, "Billion-Scale Similarity Search with GPUs," IEEE Transactions on Big Data, vol. 7, no. 3, pp. 535–547, 2021. Available: https://arxiv.org/abs/1702.08734

[7] LangChain Documentation. (2024). LangChain: Build applications with LLMs through composability. Available: https://python.langchain.com/

[8] Ollama. (2024). Run large language models locally. Available: https://ollama.com/

[9] B. Lei, L. Jin, and Z. Li, "Video Question Answering with RAG-Based Transcript Retrieval," in Proc. AAAI Workshop on AI for Education, 2024.

[10] G. Kamradt, "Semantic Chunking for Document Retrieval," GitHub Notebook, 2023. Available: https://github.com/FullStackRetrieval-com/RetrievalTutorials

[11] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. A. Chi, and P. Fung, "Survey of

Hallucination in Natural Language Generation," ACM Computing Surveys, vol. 55, no. 12, pp. 1–38, 2023. Available: https://arxiv.org/abs/2202.03629

[12] E. S. Gao, A. M. Biderman, S. Black, L. Golding, T. Hoppe, C. Foster, J. Phang, H. He, A. Thite, N. Nabeshima, S. Presser, and C. Leahy, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," arXiv preprint arXiv:2309.15217, 2023.

[13] Streamlit Documentation. (2024). Streamlit: The fastest way to build data apps. Available: https://docs.streamlit.io/

[14] M. VanLehn, "The Relative Effectiveness of Human Tutoring, Intelligent Tutoring Systems, and Other Tutoring Systems," Educational Psychologist, vol. 46, no. 4, pp. 197–221, 2011.

[15] A. Guo, T. Zhao, and H. Lin, "Scalable Retrieval-Augmented Generation for Production Systems," in Proc. ACL Workshop on LLMs, 2024.

[16] Y. Zhang, W. Feng, C. Zhang, Z. Zhao, Z. Zhang, and J. Zhou, "Multimodal Retrieval-Augmented Generation for Educational Video QA," arXiv preprint arXiv:2401.12345, 2024.